Introducción a PyTorch

¿Qué es PyTorch?

1 Una biblioteca de tensores

¿Qué es PyTorch?

1 Una biblioteca de tensores

2 Motor de diferenciación automática

¿Qué es PyTorch?

1 Una biblioteca de tensores

2 Motor de diferenciación automática

¿Qué es PyTorch?

3 Biblioteca de aprendizaje profundo

¿Qué es PyTorch?

PyTorch es un software libre y de código abierto.

Historia de PyTorch

PyTorch está basado en Torch.

https://en.wikipedia.org/wiki/Torch_(machine_learning)#/media/File:Torch_2014_logo.png

Torch fue escrito en Lua.

https://en.wikipedia.org/wiki/Lua_(programming_language)#/media/File:Lua-Logo.svg

PyTorch se basó en Torch 7.

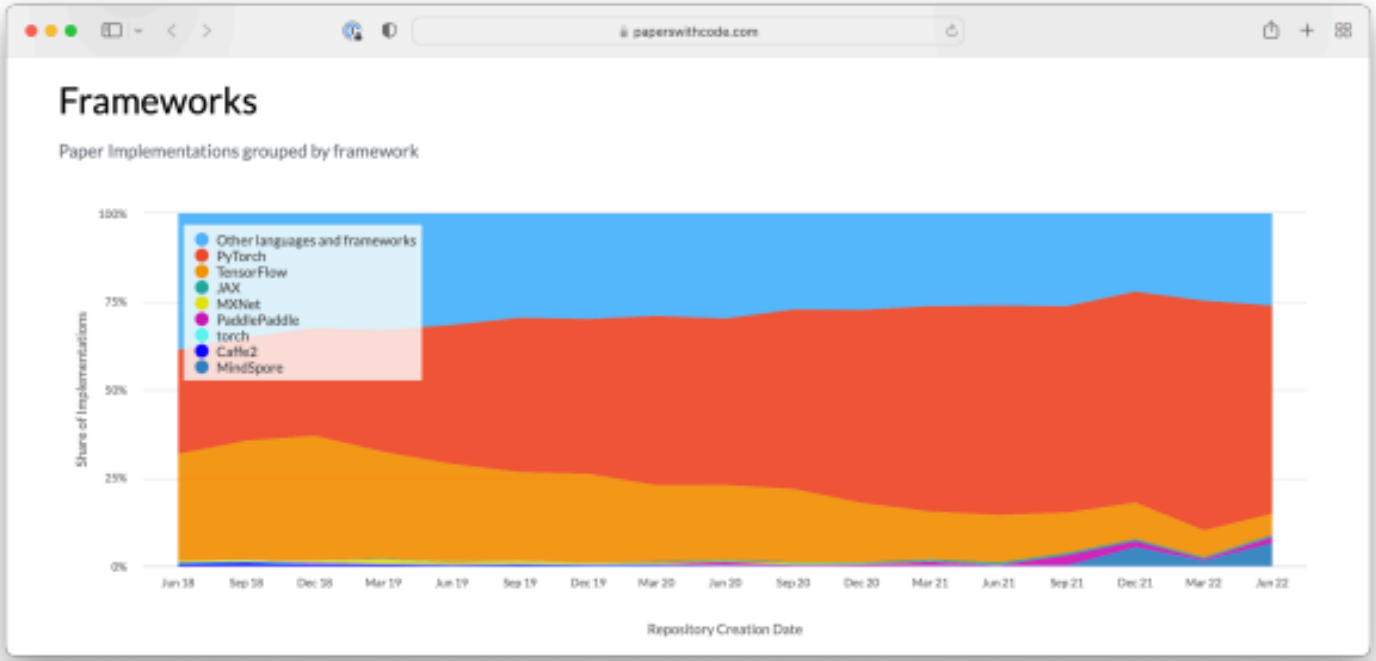
PYTORCH

2016

PyTorch

Today

PyTorch es ampliamente utilizado.



https://paperswithcode.com/trends

1 Una biblioteca de tensores

2 Motor de diferenciación automática

¿Qué es PyTorch?

3 Biblioteca de aprendizaje profundo

¿Qué son los tensores?

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

1 Una biblioteca de tensores

2 Motor de diferenciación automática

¿Qué es PyTorch?

3 Biblioteca de aprendizaje profundo

¿Qué son los tensores?

Parte 1: Tensores para datos

Una biblioteca de tensores

Matemáticamente: una generalización de vectores, matrices, etc.

Computacionalmente: un contenedor de datos.

Escalar(tensor de rango 0)

Escalar(tensor de rango 0)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

En Python puro

```
a = 1.
print(a)
```

```
1.0
```

En Pytorch

```
import torch

a = torch.tensor(1.)
print(a)
```

```
tensor(1.)
```

```
print(a.shape)
```

```
torch.Size([])
```

Vector(tensor de rango 1)

Vector(tensor de rango 1)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

En Python puro

```
a = [1., 2., 3.]
print(a)
```

```
[1.0, 2.0, 3.0]
```

En Pytorch

```
a = torch.tensor([1., 2., 3.])
print(a)
```

```
tensor([1., 2., 3.])
```

```
print(a.shape)
```

```
torch.Size([3])
```

Matriz(tensor de rango 2)

Matriz(tensor de rango 2)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```
a = torch.tensor([[1., 2., 3.],
                  [2., 3., 4.]])
a.shape
```

```
torch.Size([2, 3])
```

Matriz(tensor de rango 2)

¿Dónde hemos visto una estructura como esta antes?

```
a = torch.tensor([[1., 2., 3.],
                  [2., 3., 4.]])
a.shape
```

```
torch.Size([2, 3])
```

Matriz(tensor de rango 2)

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

¿Dónde hemos visto una estructura como esta antes?

```
a = torch.tensor([[1., 2., 3.],
                  [2., 3., 4.]])
a.shape
```

Ejemplo de entrenamiento

```
torch.Size([2, 3])
```

Matriz(tensor de rango 2)

¿Dónde hemos visto una estructura como esta antes?

Característica

```
a = torch.tensor([[1., 2., 3.],
                  [2., 3., 4.]])
a.shape
```

```
torch.Size([2, 3])
```

Imagen RGB

Image Source: https://code.tutsplus.com/tutorials/create-a-retro-crt-distortion-effect-using-rgb-shifting--active-3359

Imagen RGB

R

G

B

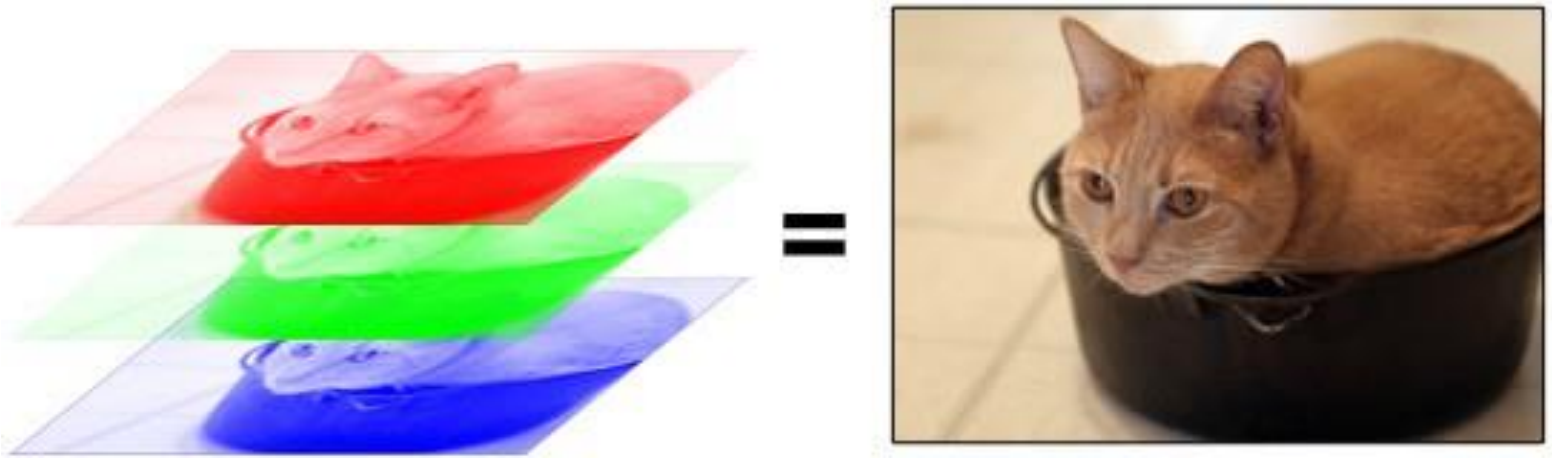


Image Source: https://code.tutsplus.com/tutorials/create-a-retro-crt-distortion-effect-using-rgb-shifting--active-3359

Imagen en color como un conjunto de matrices apiladas

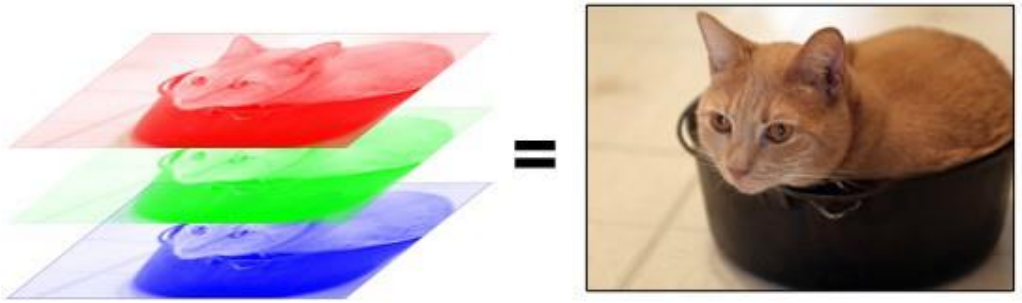


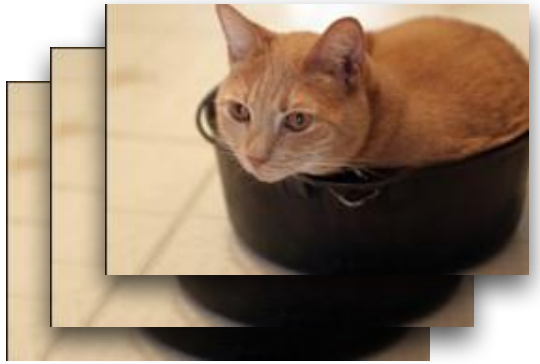
Imagen en color como un conjunto de matrices apiladas

Tensor 3D
(tensor de rango 3)

```
a = torch.tensor([[[1., 2., 3.],
                   [2., 3., 4.]],
                  [[5., 6., 7.],
                   [8., 9., 10.]]])
a.shape
```

[6]:

```
torch.Size([2, 2, 3])
```



Un conjunto de imágenes en color apiladas

Tensor 4D
(tensor de rango 4)

```
b = torch.stack((a, a))
b.shape
```

```
torch.Size([2, 2, 2, 3])
```

```
b
```

```
tensor([[[[ 1.,  2.,  3.],
          [ 2.,  3.,  4.]],

         [[ 5.,  6.,  7.],
          [ 8.,  9., 10.]]],


        [[[ 1.,  2.,  3.],
          [ 2.,  3.,  4.]],

         [[ 5.,  6.,  7.],
          [ 8.,  9., 10.]]]])
```

Biblioteca de tensores vs. Biblioteca de arreglos

¿Qué son los tensores?

Parte 2: Tensores y bibliotecas de arreglos

1 Una biblioteca de tensores

2 Motor de diferenciación automática

¿Qué es PyTorch?

3 Biblioteca de aprendizaje profundo

Una biblioteca de tensores

Matemáticamente: una generalización de vectores, matrices, etc.

Una biblioteca de tensores

Biblioteca de tensores == biblioteca de arreglos

Una biblioteca de tensores

```
torch.tensor ≈ numpy.array
```

Una biblioteca de tensores

torch.tensor ≈ numpy.array

+ Soporte para GPU

Una biblioteca de tensores

`torch.tensor ≈ numpy.array`

+ Soporte para GPU

+ Soporte para diferenciación automática

Listas de Python vs. Arreglos/Tensores

Listas de Python vs. Arreglos/Tensores

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Lista

+ Pueden almacenar tipos heterogéneos(mezcla de str, float, etc.)

+ Se pueden agregar o eliminar elementos fácilmente

- Los cálculos numéricos son lentos

Listas de Python vs. Arreglos/Tensores

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

Lista

+ Pueden almacenar tipos heterogéneos(mezcla de str, float, etc.)

+ Se pueden agregar o eliminar elementos fácilmente

- Los cálculos numéricos son lentos

Tensor

- Todos los elementos deben tener el mismo tipo(por ejemplo, int, float)

- No se pueden agregar ni eliminar elementos

+ Soporta muchas operaciones matemáticas

+ Los cálculos numéricos son rápidos

¿Cómo usamos tensores en PyTorch?

¿Cómo usamos tensores en PyTorch?

Mis 10 funciones y métodos de tensores favoritos

Mis 10 funciones y métodos de tensores favoritos

(Y más adelante presentaremos más, poco a poco...)

1. Creación de tensores

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
import torch


m = torch.tensor([[1., 2., 3.],
                  [4., 5., 6.]])
print(v)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

2. Verificación de la forma (shape)

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

```python
print(m.shape)
```

```
torch.Size([2, 3])
```

3. Verificación del rango / número de dimensiones

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

```python
print(m.ndim)
```

```
2
```

4. Verificación del tipo de dato

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

```python
print(m.dtype)
```

```
torch.float32
```

4. Verificación del tipo de dato

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
other_m = torch.tensor([[1, 2, 3],
                        [4, 5, 6]])
print(other_m)
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

4. Verificación del tipo de dato

UNIVERSIDAD EAFIT | Escuela de Ciencias Aplicadas e Ingeniería

```
other_m = torch.tensor([[1, 2, 3],
                        [4, 5, 6]])
print(other_m)
```

```
tensor([[1, 2, 3],
        [4, 5, 6]])
```

```
print(other_m.dtype)
```

```
torch.int64
```

5. Creación de un tensor a partir de arreglos de NumPy

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
import numpy as np

np_ary = np.array([1., 2., 3.])
m2 = torch.from_numpy(np_ary)

print(m2)
```

```
tensor([1., 2., 3.], dtype=torch.float64)
```

```python
m3 = torch.tensor(np_ary)

print(m3)
```

```
tensor([1., 2., 3.], dtype=torch.float64)
```

6. Cambio de tipos de datos

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m2.dtype)
```

```
torch.float64
```

6. Cambio de tipos de datos

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m2.dtype)
```

```
torch.float64
```

```python
m2 = m2.to(torch.float32)
print(m2.dtype)
```

```
torch.float32
```

7. Verificación del tipo de dispositivo

```
m2.device
```

```
device(type='cpu')
```

8. Cambio de la forma del tensor

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

```python
print(m.view(3, 2))
```

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
```

8. Cambio de la forma del tensor

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m.view(3, 2))
```

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
```

```python
print(m.view(-1, 2))
```

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
```

8. Cambio de la forma del tensor

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m.view(3, 2))
```

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
```

[69]:

```python
print(m.view(-1, 2))
```

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
```

[70]:

```python
print(m.view(3, -1))
```

```
tensor([[1., 2.],
        [3., 4.],
        [5., 6.]])
```

9. Transposición de un tensor

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

```python
print(m.T)
```

```
tensor([[1., 4.],
        [2., 5.],
        [3., 6.]])
```

9. Transposición de un tensor

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

1.	2.	3.
4.	5.	6.

```python
print(m.T)
```

```
tensor([[1., 4.],
        [2., 5.],
        [3., 6.]])
```

9. Transposición de un tensor


```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

```python
print(m.T)
```

```
tensor([[1., 4.],
        [2., 5.],
        [3., 6.]])
```

1.	4.
2.	5.
3.	6.

10. Multiplicación de matrices

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

10. Multiplicación de matrices

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

```python
print(m.T)
```

```
tensor([[1., 4.],
        [2., 5.],
        [3., 6.]])
```

10. Multiplicación de matrices

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
print(m)
```

```
tensor([[1., 2., 3.],
        [4., 5., 6.]])
```

```python
print(m.T)
```

```
tensor([[1., 4.],
        [2., 5.],
        [3., 6.]])
```

```python
m.matmul(m.T)
```

[73]:

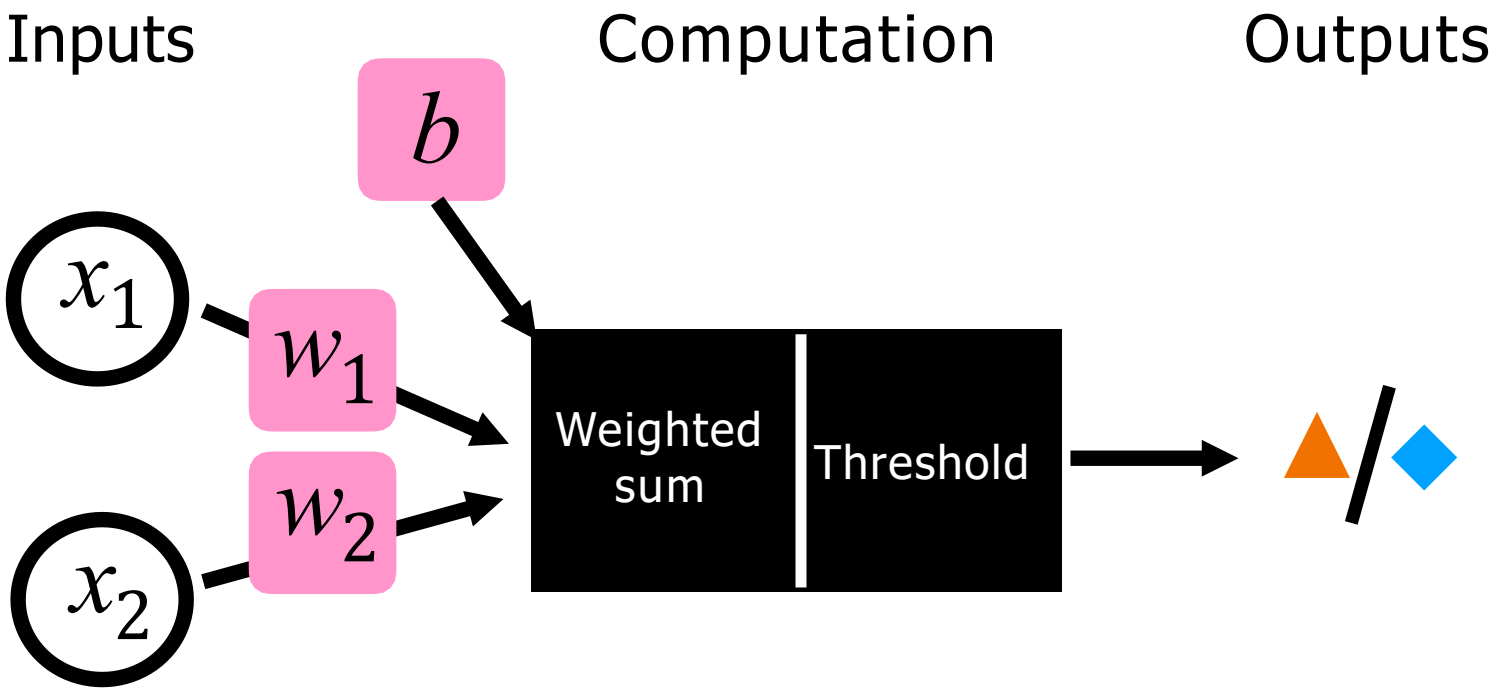
```
tensor([[14., 32.],
        [32., 77.]])
```

Usando conceptos del álgebra lineal para hacer nuestro código más eficiente...

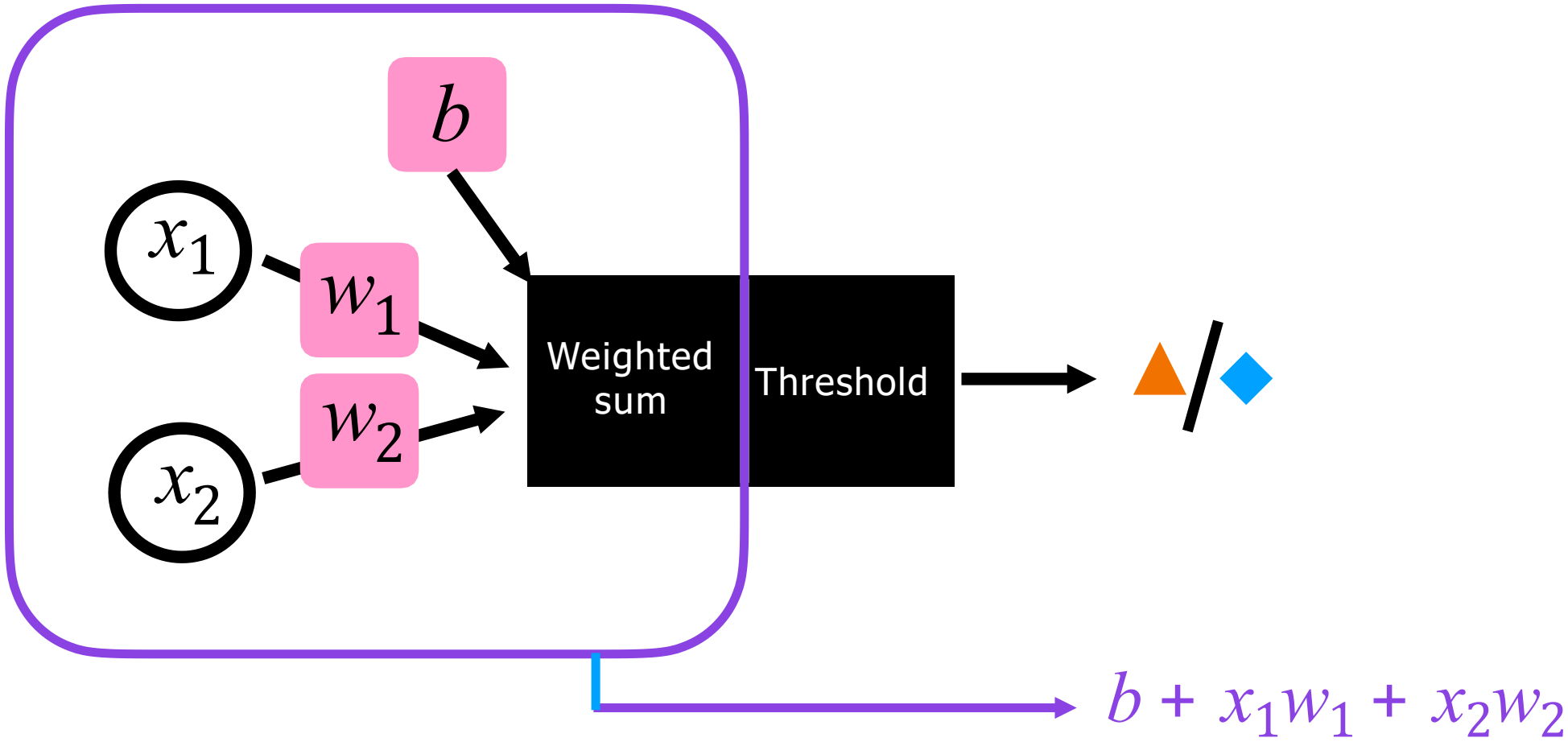
Mejorando la eficiencia del código con álgebra lineal

Parte 1: De bucles a productos punto

El perceptrón



El perceptrón



1 ejemplo de entrenamiento
con 2 valores de características

$$z = b + x_1 \times w_1 + x_2 \times w_2$$

1 ejemplo de entrenamiento
con 2 valores de características

$$z = b + w_1 \times x_1 + w_2 \times x_2$$

Forma General

$$z = b + \sum_{j=1}^{m} x_j w_j$$

Suma ponderada en Python puro

$$z = b + \sum_{j=1}^{m} x_j w_j$$

```python
b = 0.
x = [1.2, 2.2]
w = [3.3, 4.3]

output = b
for x_j, w_j in zip(x, w):
    output += x_j * w_j

print(output)
```

```
13.42
```

Suma ponderada en PyTorch usando álgebra lineal

$$z = b + \sum_{j=1}^{m} x_j w_j$$

$$= b + \mathbf{xw}$$

Producto punto entre dos vectores

Producto punto entre 2 vectores

$$z = b + \mathbf{x}\mathbf{w}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$

Producto punto entre 2 vectores

$$z = b + \boxed{\mathbf{x}^\top}\mathbf{w}$$

La transposición T no es necesaria, pero es más intuitiva cuando hablemos de multiplicación de matrices más adelante.

$$\boxed{\mathbf{x}^\top = [x_1\, x_2 \;\ldots\; x_m]} \qquad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$

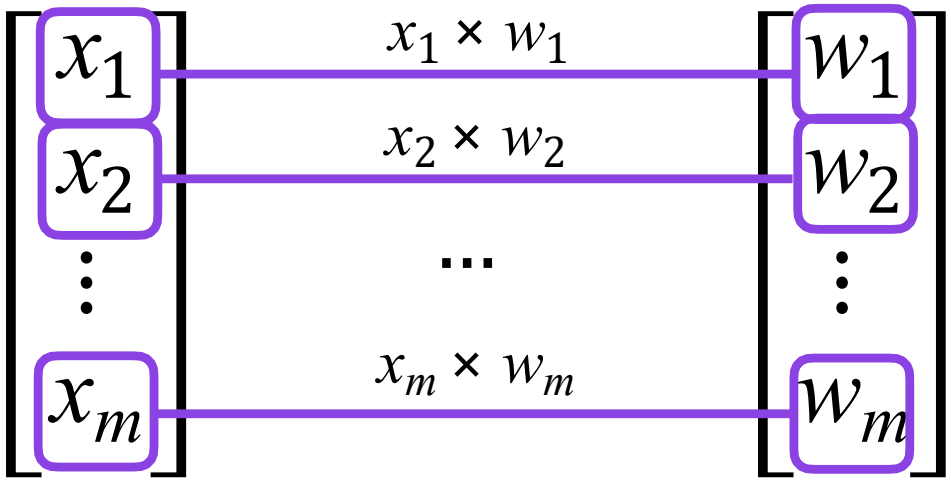
Producto punto entre 2 vectores

$$z = b + \mathbf{x}\mathbf{w}$$

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$

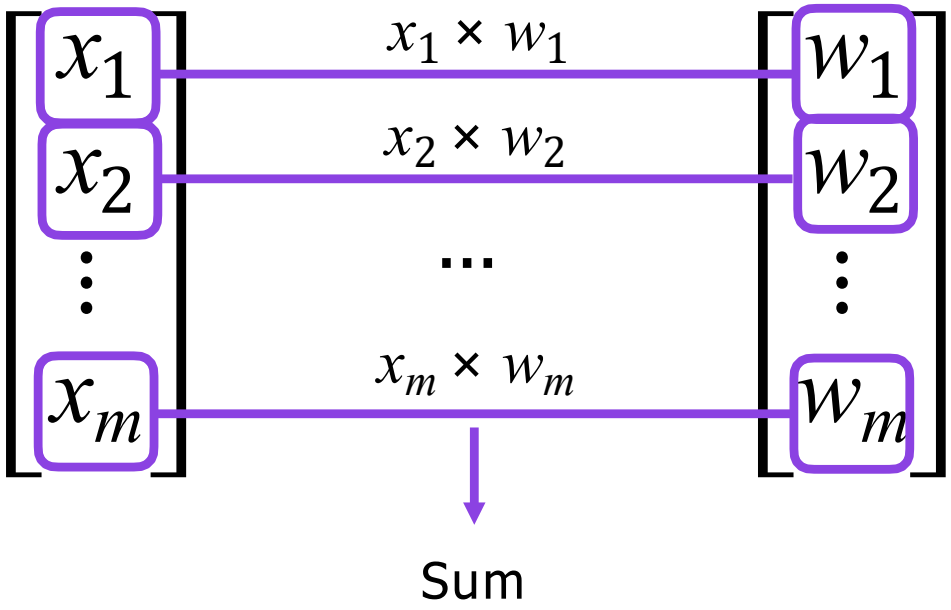
Producto punto entre 2 vectores

$$z = b + \boxed{\mathbf{xw}}$$



Producto punto entre 2 vectores

$$z = b + \boxed{\mathbf{xw}}$$



Producto punto en PyTorch

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
import torch

b = torch.tensor([0.])
x = torch.tensor([1.2, 2.2])
w = torch.tensor([3.3, 4.3])

x.dot(w) + b
```

```
tensor([13.4200])
```

Producto punto en PyTorch

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
import torch

b = torch.tensor([0.])
x = torch.tensor([1.2, 2.2])
w = torch.tensor([3.3, 4.3])

x.dot(w) + b
```

```
tensor([13.4200])
```

```python
b = 0.
x = [1.2, 2.2]
w = [3.3, 4.3]

output = b
for x_j, w_j in zip(x, w):
    output += x_j * w_j

print(output)
```

```
13.42
```

El producto punto es más compacto, ¿pero también es más rápido?

Producto punto en PyTorch

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
import torch

b = torch.tensor([0.])
x = torch.tensor([1.2, 2.2])
w = torch.tensor([3.3, 4.3])

x.dot(w) + b
```

```
tensor([13.4200])
```

```python
b = 0.
x = [1.2, 2.2]
w = [3.3, 4.3]

output = b
for x_j, w_j in zip(x, w):
    output += x_j * w_j

print(output)
```

```
13.42
```

Producto punto en PyTorch

El producto punto es más compacto, ¿pero también es más rápido?

```
output = b
for x_j, w_j in zip(x, w):
    output += x_j * w_j
```

```
output = x.dot(w) + b
```

Evaluación del rendimiento del producto punto

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
def plain_python(x, w, b):
    output = b
    for x_j, w_j in zip(x, w):
        output += x_j * w_j
    return output
```

Evaluación del rendimiento del producto punto

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
def plain_python(x, w, b):
    output = b
    for x_j, w_j in zip(x, w):
        output += x_j * w_j
    return output
```

```python
import random


random.seed(123)

b = 0.
x = [random.random() for _ in range(1000)]
w = [random.random() for _ in range(1000)]
```

Evaluación del rendimiento del producto punto

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
def plain_python(x, w, b):
    output = b
    for x_j, w_j in zip(x, w):
        output += x_j * w_j
    return output
```

```python
import random


random.seed(123)

b = 0.
x = [random.random() for _ in range(1000)]
w = [random.random() for _ in range(1000)]
```

```python
%timeit plain_python(x, w, b)
```

```
35.3 µs ± 175 ns per loop (mean ± std. dev. of
7 runs, 10,000 loops each)
```

Evaluación del rendimiento del producto punto

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
def pytorch_dot(x, w, b):
    return x.dot(w) + b
```

Evaluación del rendimiento del producto punto

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
def pytorch_dot(x, w, b):
    return x.dot(w) + b
```

```python
t_b = torch.tensor(b)
t_x = torch.tensor(x)
t_w = torch.tensor(w)
```

Evaluación del rendimiento del producto punto

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
def pytorch_dot(x, w, b):
    return x.dot(w) + b
```

```python
t_b = torch.tensor(b)
t_x = torch.tensor(x)
t_w = torch.tensor(w)
```

```python
%timeit pytorch_dot(t_x, t_w, t_b)
```

```
1.42 µs ± 18.6 ns per loop (mean ± std. dev. o
f 7 runs, 1,000,000 loops each)
```

Evaluación del rendimiento del producto punto

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

A 25x speed-up!

```
%timeit pytorch_dot(t_x, t_w, t_b)
```

```
1.42 µs ± 18.6 ns per loop (mean ± std. dev. o
f 7 runs, 1,000,000 loops each)
```

```
%timeit plain_python(x, w, b)
```

```
35.3 µs ± 175 ns per loop (mean ± std. dev. of
7 runs, 10,000 loops each)
```

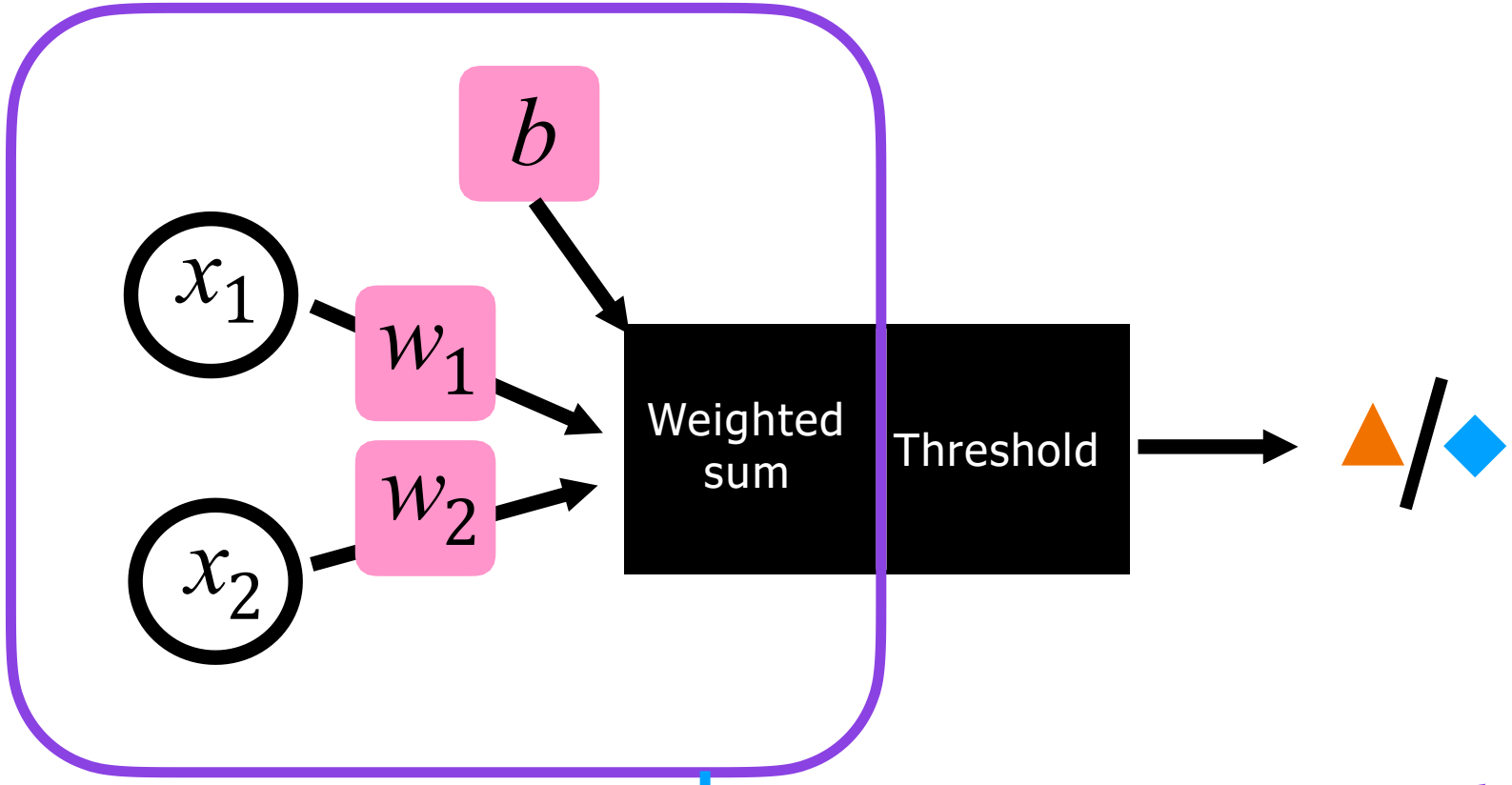
¿Qué pasa con múltiples ejemplos de entrenamiento?

¡Echemos un vistazo a la multiplicación de matrices!

Mejorando la eficiencia del código con álgebra lineal

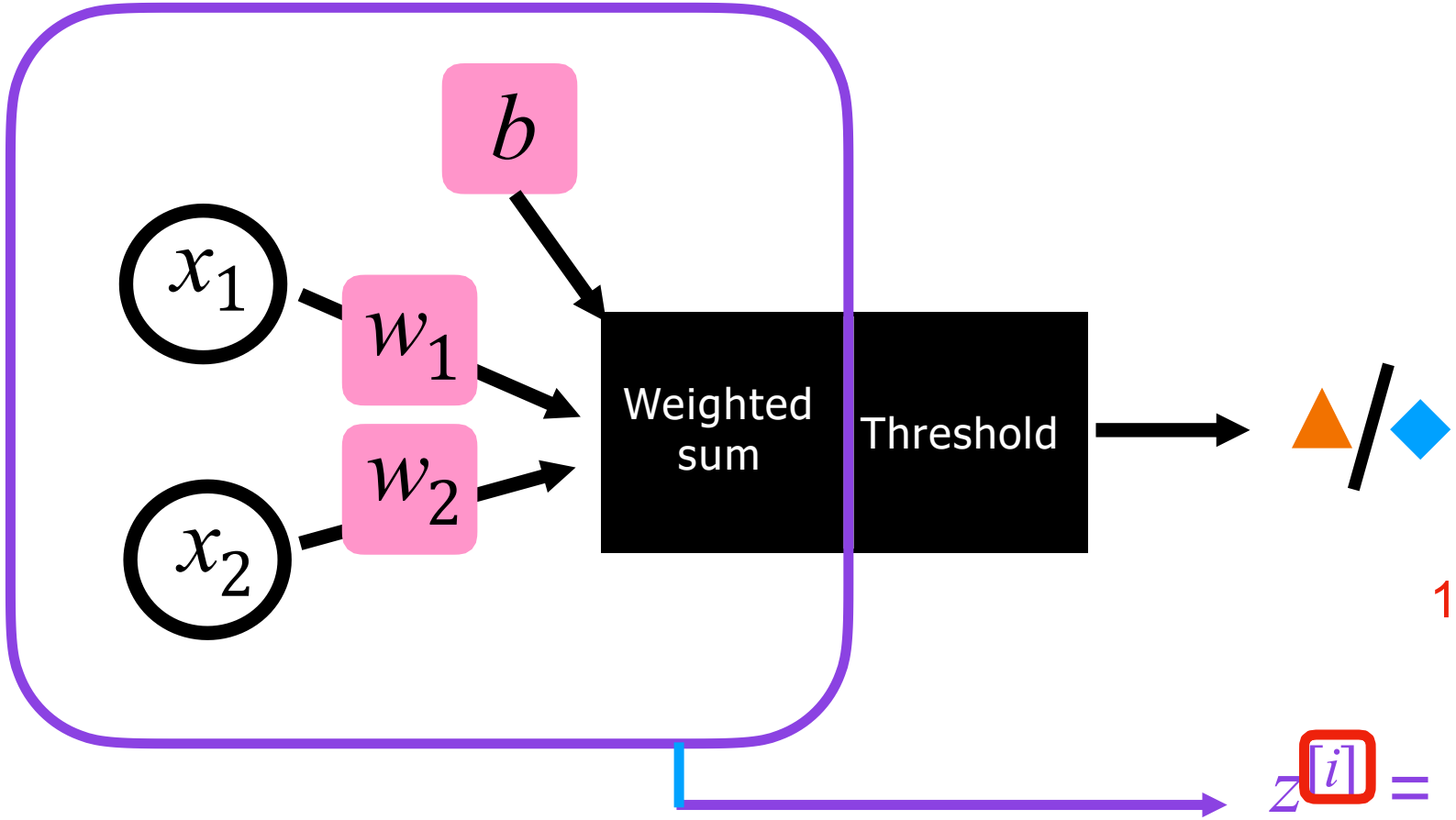
Parte 2: Manejo de múltiples ejemplos de entrenamiento mediante multiplicación de matrices

El perceptrón



$$z = b + \sum_{j=1}^{m} x_j w_j$$

El perceptrón



1 training example

$$z^{[i]} = b + \sum_{j=1}^{m} x_j^{[i]} w_j$$

Procesamiento de múltiples ejemplos de entrenamiento

$$z^{[1]} = b + \sum_{j=1}^{m} x_j^{[1]} w_j$$

$$z^{[2]} = b + \sum_{j=1}^{m} x_j^{[2]} w_j$$

$$\cdots$$

$$z^{[n]} = b + \sum_{j=1}^{m} x_j^{[n]} w_j$$

Procesamiento de múltiples ejemplos de entrenamiento

$$z^{[1]} = b + \mathbf{x}^{[1]}\mathbf{w}$$

$$z^{[2]} = b + \mathbf{x}^{[2]}\mathbf{w}$$

$$\vdots$$

$$z^{[n]} = b + \mathbf{x}^{[n]}\mathbf{w}$$

Ejemplos de entrenamiento como una matriz

$$z^{[1]} = b + \mathbf{x}^{[1]}\mathbf{w}$$

$$z^{[2]} = b + \mathbf{x}^{[2]}\mathbf{w}$$

$$\cdots$$

$$z^{[n]} = b + \mathbf{x}^{[n]}\mathbf{w}$$

Ejemplos de entrenamiento en forma de matriz

$$z^{[1]} = b + \mathbf{x}^{[1]}\mathbf{w}$$

$$z^{[2]} = b + \mathbf{x}^{[2]}\mathbf{w}$$

$$\cdots$$

$$z^{[n]} = b + \mathbf{x}^{[n]}\mathbf{w}$$

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{[1]} \\ \mathbf{x}^{[2]} \\ \vdots \\ \mathbf{x}^{[n]} \end{bmatrix}$$

Ejemplos de entrenamiento en forma de matriz

$$\mathbf{X} = \begin{bmatrix} \mathbf{x}^{[1]} \\ \mathbf{x}^{[2]} \\ \vdots \\ \mathbf{x}^{[n]} \end{bmatrix} = \begin{bmatrix} x_1^{[1]}\, x_2^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix}$$

Ejemplos de entrenamiento en forma de matriz

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} x_2^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]} x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]} x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$

n training examples

1 weight vector

Multiplicación de matrices

Requisito: el número de columnas y filas debe coincidir

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} x_2^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]} x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]} x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix}$$

n training examples

1 weight vector

Multiplicación de matrices

Producto punto

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_2^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix} \qquad \mathbf{Xw} = \begin{bmatrix} \square \\ \\ \\ \end{bmatrix}$$

n training examples

1 weight vector

Result

Multiplicación de matrices

Producto punto

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_2^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix} \qquad \mathbf{Xw} = \begin{bmatrix} \square \\ \square \\ \\ \end{bmatrix}$$

n training examples

1 weight vector

Result

Multiplicación de matrices

Producto punto

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_2^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \qquad \mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_m \end{bmatrix} \qquad \mathbf{Xw} = \begin{bmatrix} \square \\ \square \\ \vdots \\ \square \end{bmatrix}$$

n training examples

1 weight vector

Result

¡Hora de la evaluación de rendimiento!

Multiplicación de una matriz y un vector en Python puro

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
b = 0.
X = [[1.2, 2.2],
     [4.4, 5.5]]
w = [3.3, 4.3]


outputs = []
for x in X:
    output = b
    for x_j, w_j in zip(x, w):
        output += x_j * w_j
    outputs.append(output)

outputs
```

```
[13.42, 38.17]
```

Multiplicación de una matriz y un vector en PyTorch

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
b = 0.
X = [[1.2, 2.2],
     [4.4, 5.5]]
w = [3.3, 4.3]


outputs = []
for x in X:
    output = b
    for x_j, w_j in zip(x, w):
        output += x_j * w_j
    outputs.append(output)

outputs
```

```
[13.42, 38.17]
```

```python
import torch

b = torch.tensor([0.])
X = torch.tensor(
    [[1.2, 2.2],
     [4.4, 5.5]]
)
w = torch.tensor([3.3, 4.3])

X.matmul(w) + b
```

```
tensor([13.4200, 38.1700])
```

Evaluación del rendimiento de la implementación en Python puro

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
import random

random.seed(123)

b = 0.
X = [[random.random() for _ in range(1000)] # 500 rows
     for i in range(500)]
w = [random.random() for _ in range(1000)]
```

```python
def plain_python(X, w, b):
    outputs = []
    for x in X:
        output = b
        for x_j, w_j in zip(x, w):
            output += x_j * w_j
        outputs.append(output)
    return outputs
```

```python
%timeit plain_python(X, w, b)
```

```
18 ms ± 205 µs per loop (mean ± std. dev. of 7 runs, 100
loops each)
```

Evaluación del rendimiento de la implementación en Python puro

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
t_b = torch.tensor(b)
t_X = torch.tensor(X)
t_w = torch.tensor(w)
```

```python
def pytorch_implementation(X, w, b):
    return X.matmul(w) + b
```

```python
%timeit pytorch_implementation(t_X, t_w, t_b)
```

```
23 µs ± 1.64 µs per loop (mean ± std. dev. of 7 runs,
10,000 loops each)
```

approx. 1000x speed-up!

A continuación: Multiplicación de matrices con 2 matrices

Mejorando la eficiencia del código con álgebra lineal

Parte 3: Multiplicación de dos matrices

Vista previa

Multiplicación de matrices con una matriz de pesos

Multiplicación de matrices con una matriz de pesos

Requisito: el número de columnas y filas debe coincidir

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]} x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]} x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \qquad \mathbf{W} = \begin{bmatrix} w_1^{(1)} w_1^{(1)}, & \dots & , w_m^{(1)} \\ w_1^{(2)} w_2^{(2)}, & \dots & , w_m^{(2)} \\ & \vdots & \\ w_1^{(h)} w_2^{(h)}, & \dots & , w_m^{(h)} \end{bmatrix}$$

n training examples

h weight vectors

Multiplicación de matrices con una matriz de pesos

Requisito: el número de columnas y filas debe coincidir

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]} x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]} x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]} x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \qquad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)} w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)} w_2^{(2)}, & \dots & , w_2^{(h)} \\ & \vdots & \\ w_m^{(1)} w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix}$$

n training examples

h weight vectors

Multiplicación de matrices con una matriz de pesos

UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)}\, w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)}\, w_2^{(2)}, & \dots & , w_2^{(h)} \\ & \vdots & \\ w_m^{(1)}\, w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix} \quad \mathbf{X}\mathbf{W}^\top = \begin{bmatrix} \square & & \\ & & \\ & & \\ & & \end{bmatrix}$$

Multiplicación de matrices con una matriz de pesos

UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)}\, w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)}\, w_2^{(2)}, & \dots & , w_2^{(h)} \\ & \vdots & \\ w_m^{(1)}\, w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix} \quad \mathbf{X}\mathbf{W}^\top = \begin{bmatrix} \square & \\ \square & \\ & \\ & \end{bmatrix}$$

Multiplicación de matrices con una matriz de pesos

UNIVERSIDAD EAFIT

Escuela de Ciencias Aplicadas e Ingeniería

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)}\, w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)}\, w_2^{(2)}, & \dots & , w_2^{(h)} \\ & \vdots & \\ w_m^{(1)}\, w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix} \quad \mathbf{X}\mathbf{W}^\top = \begin{bmatrix} \square & & \\ \square & & \\ \vdots & & \\ \square & & \end{bmatrix}$$

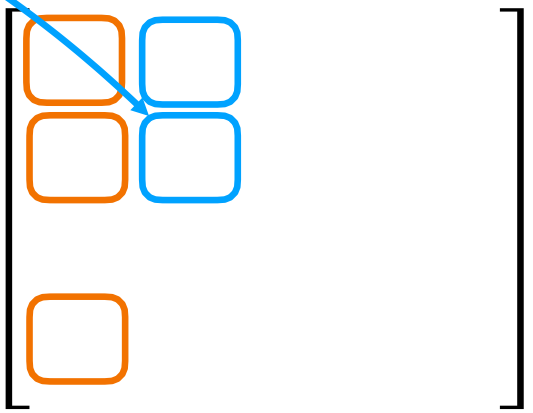
Multiplicación de matrices con una matriz de pesos

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)}\, w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)}\, w_2^{(2)}, & \dots & , w_2^{(h)} \\ & \vdots & \\ w_m^{(1)}\, w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix} \quad \mathbf{X}\mathbf{W}^\top = \begin{bmatrix} \square & \square & \\ \square & & \\ \vdots & & \\ \square & & \end{bmatrix}$$

Multiplicación de matrices con una matriz de pesos

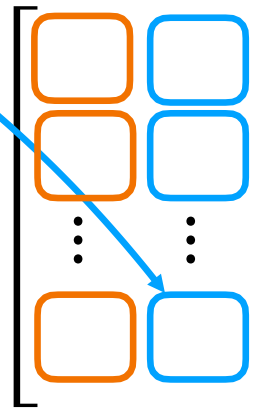
$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)}\, w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)}\, w_2^{(2)}, & \dots & , w_2^{(h)} \\ & \vdots & \\ w_m^{(1)}\, w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix} \quad \mathbf{X}\mathbf{W}^\top =$$



Multiplicación de matrices con una matriz de pesos

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)}\, w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)}\, w_2^{(2)}, & \dots & , w_2^{(h)} \\ & \vdots & \\ w_m^{(1)}\, w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix}$$

$\mathbf{X}\mathbf{W}^\top =$



Multiplicación de matrices con una matriz de pesos

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)}\, w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)}\, w_2^{(2)}, & \dots & , w_2^{(h)} \\ & \vdots & \\ w_m^{(1)}\, w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix} \quad \mathbf{X}\mathbf{W}^\top = \begin{bmatrix} \square & \square & \cdots & \square \\ \square & \square & \cdots & \square \\ \vdots & \vdots & & \vdots \\ \square & \square & \cdots & \square \end{bmatrix}$$

Dimensiones de salida

$$\mathbf{X} = \begin{bmatrix} x_1^{[1]}\, x_1^{[1]}, & \dots & , x_m^{[1]} \\ x_1^{[2]}\, x_2^{[2]}, & \dots & , x_m^{[2]} \\ & \vdots & \\ x_1^{[n]}\, x_2^{[n]}, & \dots & , x_m^{[n]} \end{bmatrix} \quad \mathbf{W}^\top = \begin{bmatrix} w_1^{(1)}\, w_1^{(2)}, & \dots & , w_1^{(h)} \\ w_2^{(1)}\, w_2^{(2)}, & \dots & , w_2^{(h)} \\ & \vdots & \\ w_m^{(1)}\, w_m^{(2)}, & \dots & , w_m^{(h)} \end{bmatrix}$$

$$\mathbf{X}\mathbf{W}^\top = \begin{bmatrix} \square & \square & \cdots & \square \\ \square & \square & \cdots & \square \\ \vdots & \vdots & & \vdots \\ \square & \square & \cdots & \square \end{bmatrix}$$

Un ejemplo en PyTorch

UNIVERSIDAD EAFIT
Escuela de Ciencias Aplicadas e Ingeniería

```python
import torch


X = torch.rand(100, 10)
W = torch.rand(50, 10)

R = torch.matmul(X, W.T)
```

```python
R.shape
```

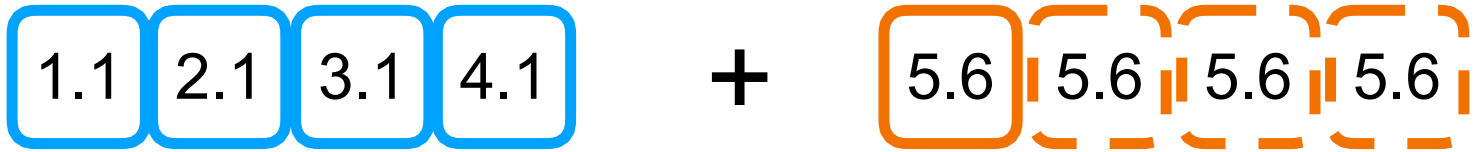
```
torch.Size([100, 50])
```

Broadcasting – Cálculos con formas de tensores desiguales

Mejorando la eficiencia del código con álgebra lineal

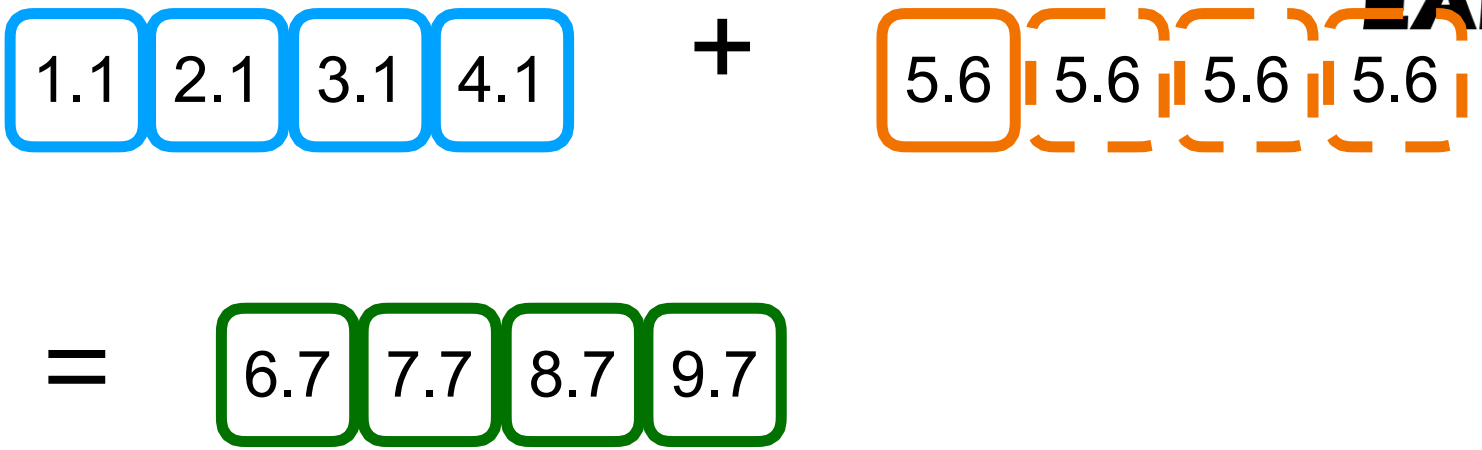
Parte 4: Broadcasting – Cálculos con formas de tensores desiguales

1.1 2.1 3.1 4.1 + 5.6


1.1
2.1
3.1
4.1
+
5.6
5.6
5.6
5.6

$$\begin{bmatrix} 1.1 & 2.1 & 3.1 & 4.1 \end{bmatrix} + \begin{bmatrix} 5.6 & 5.6 & 5.6 & 5.6 \end{bmatrix}$$

$$= \begin{bmatrix} 6.7 & 7.7 & 8.7 & 9.7 \end{bmatrix}$$



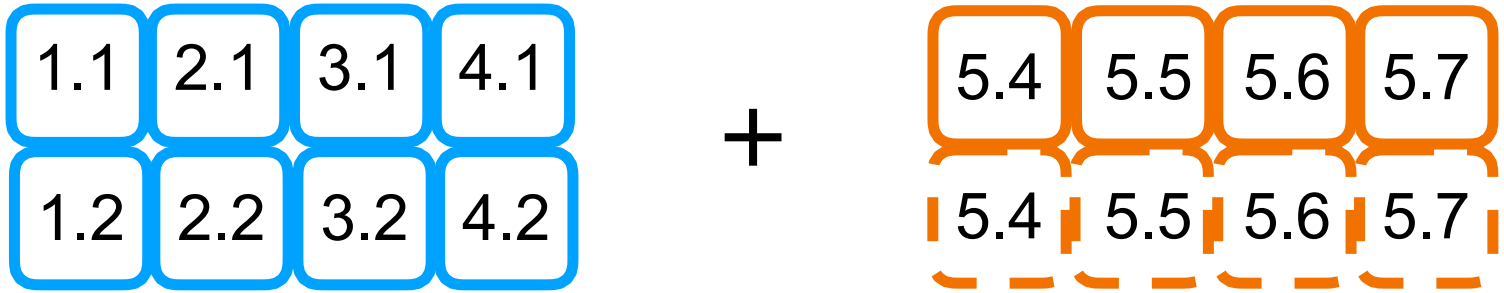

```python
a = torch.tensor([1.1, 2.1, 3.1, 4.1])
b = torch.tensor([5.6])

a + b
```

```
tensor([6.7000, 7.7000, 8.7000, 9.7000])
```

1.1	2.1	3.1	4.1
1.2	2.2	3.2	4.2

+

5.4	5.5	5.6	5.7


1.1
2.1
3.1
4.1
1.2
2.2
3.2
4.2
+
5.4
5.5
5.6
5.7
5.4
5.5
5.6
5.7

1.1	2.1	3.1	4.1
1.2	2.2	3.2	4.2

+

5.4	5.5	5.6	5.7
5.4	5.5	5.6	5.7

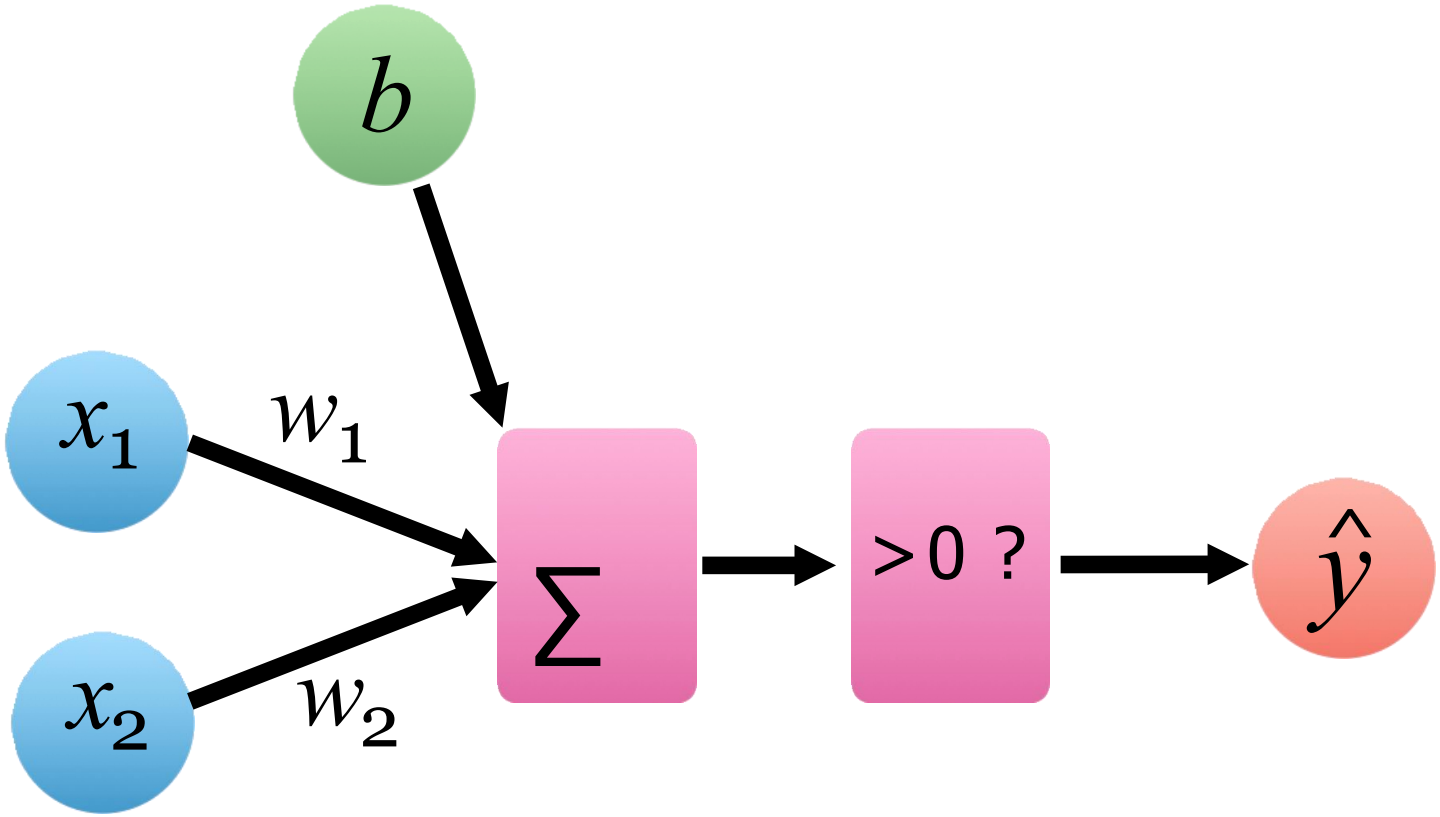
```python
A = torch.tensor([[1.1, 2.1, 3.1, 4.1],
                  [1.2, 2.2, 3.2, 4.2]])

b = torch.tensor([5.4, 5.5, 5.6, 5.7])

A + b
```

```
tensor([[6.5000, 7.6000, 8.7000, 9.8000],
        [6.6000, 7.7000, 8.8000, 9.9000]])
```

A continuación: Depuración de código

Visualización de modelos predictivos como grafos computacionales

El modelo de perceptrón



```python
class Perceptron:
    def __init__(self, num_features):
        self.num_features = num_features
        self.weights = torch.zeros(num_features)
        self.bias = torch.tensor(0.)

    def forward(self, x):
        weighted_sum_z = torch.dot(x, self.weights) + self.bias

        if weighted_sum_z > 0.:
            prediction = torch.tensor(1.)
        else:
            prediction = torch.tensor(0.)

        return prediction

    def update(self, x, true_y):
        prediction = self.forward(x)
        error = true_y - prediction

        # update
        self.bias += error
        self.weights += error * x

        return error
```

El modelo de perceptrón

b

w_1

w_2

Entradas de datos

b

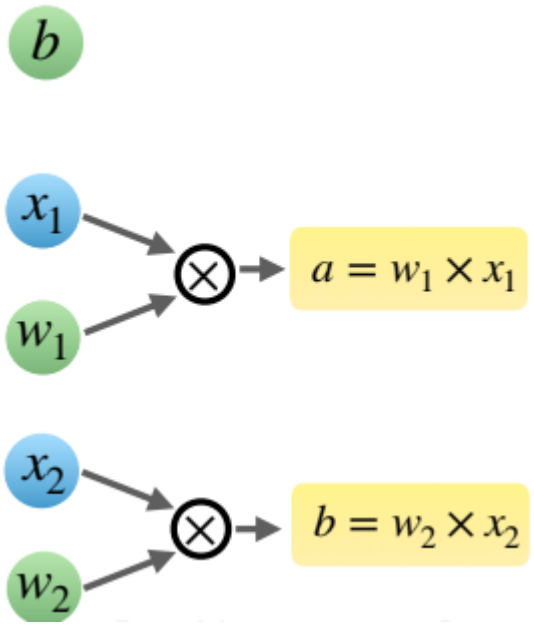
x_1

w_1

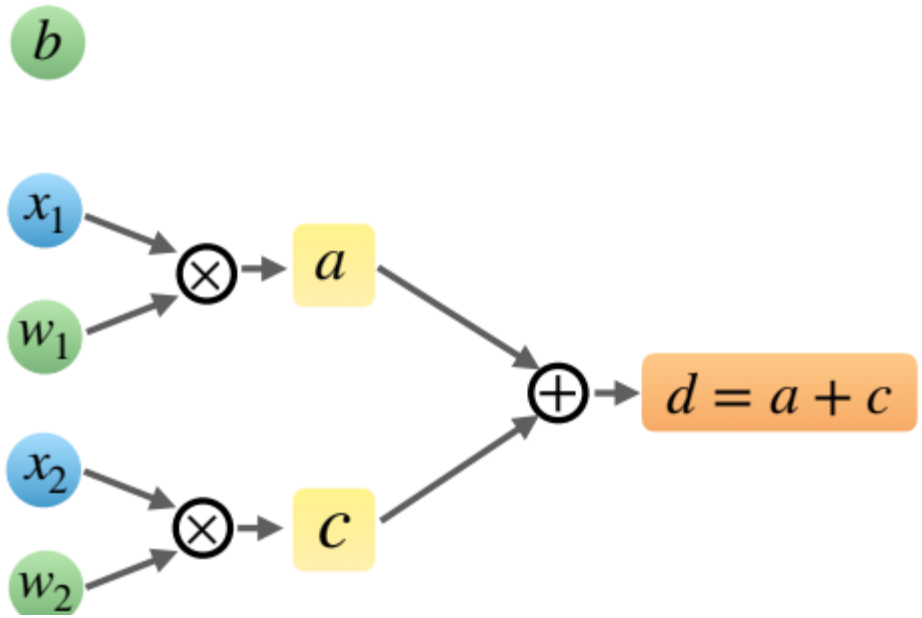
x_2

w_2

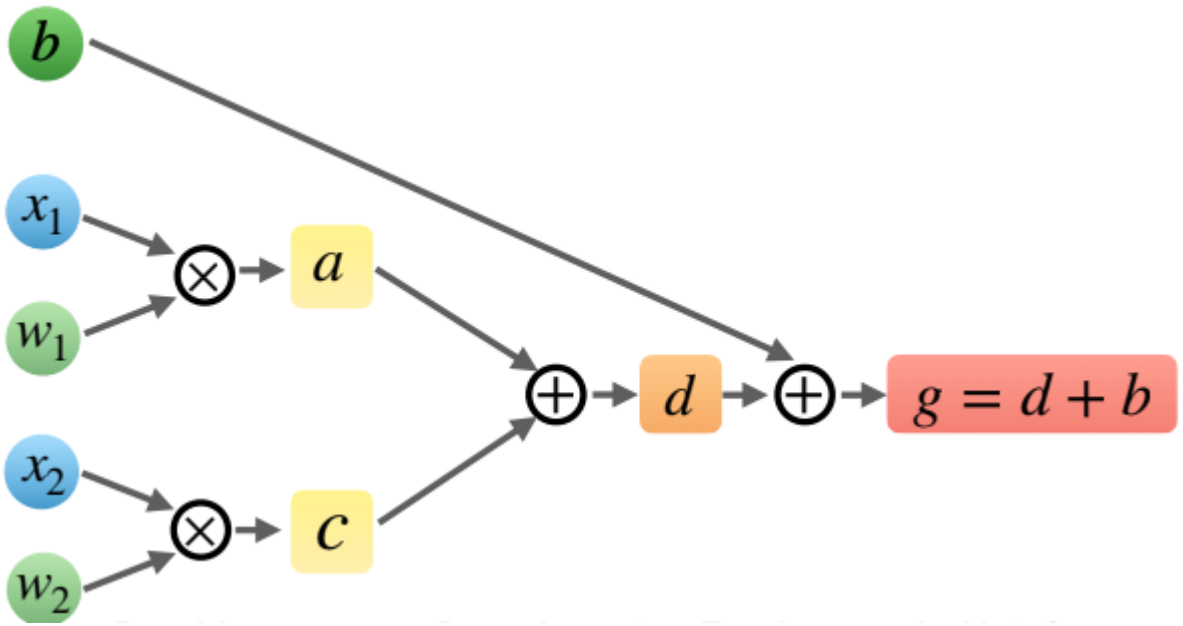
Suma ponderada



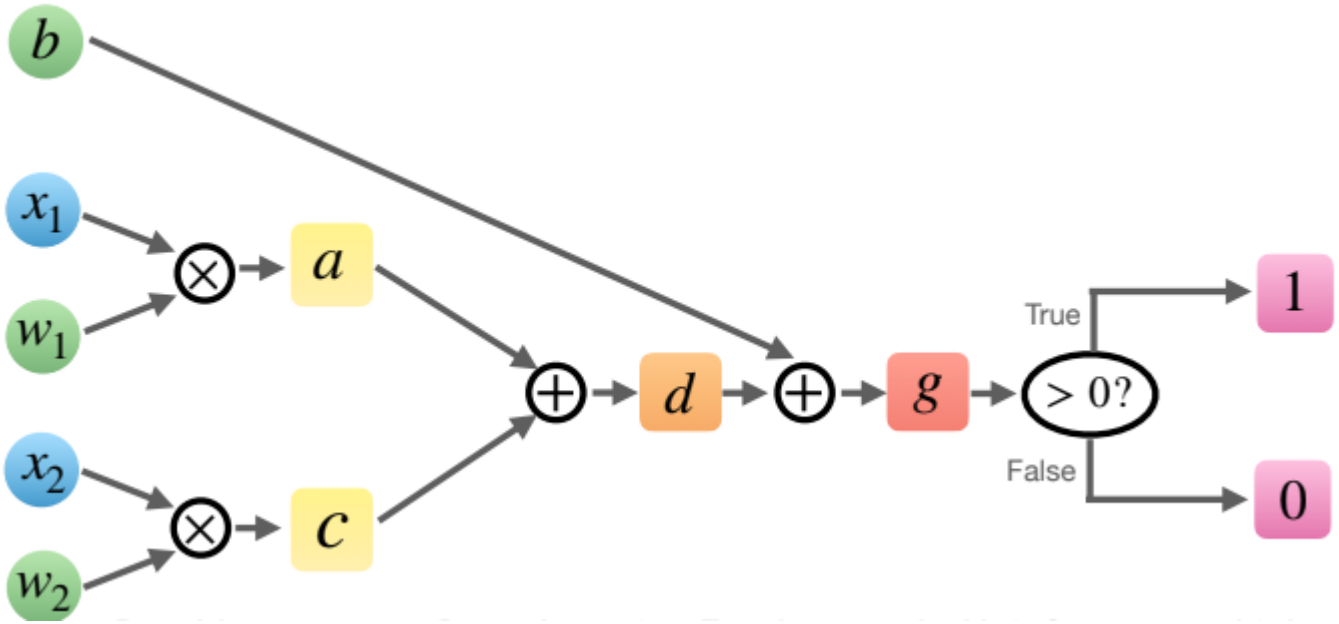
Suma ponderada



Suma ponderada

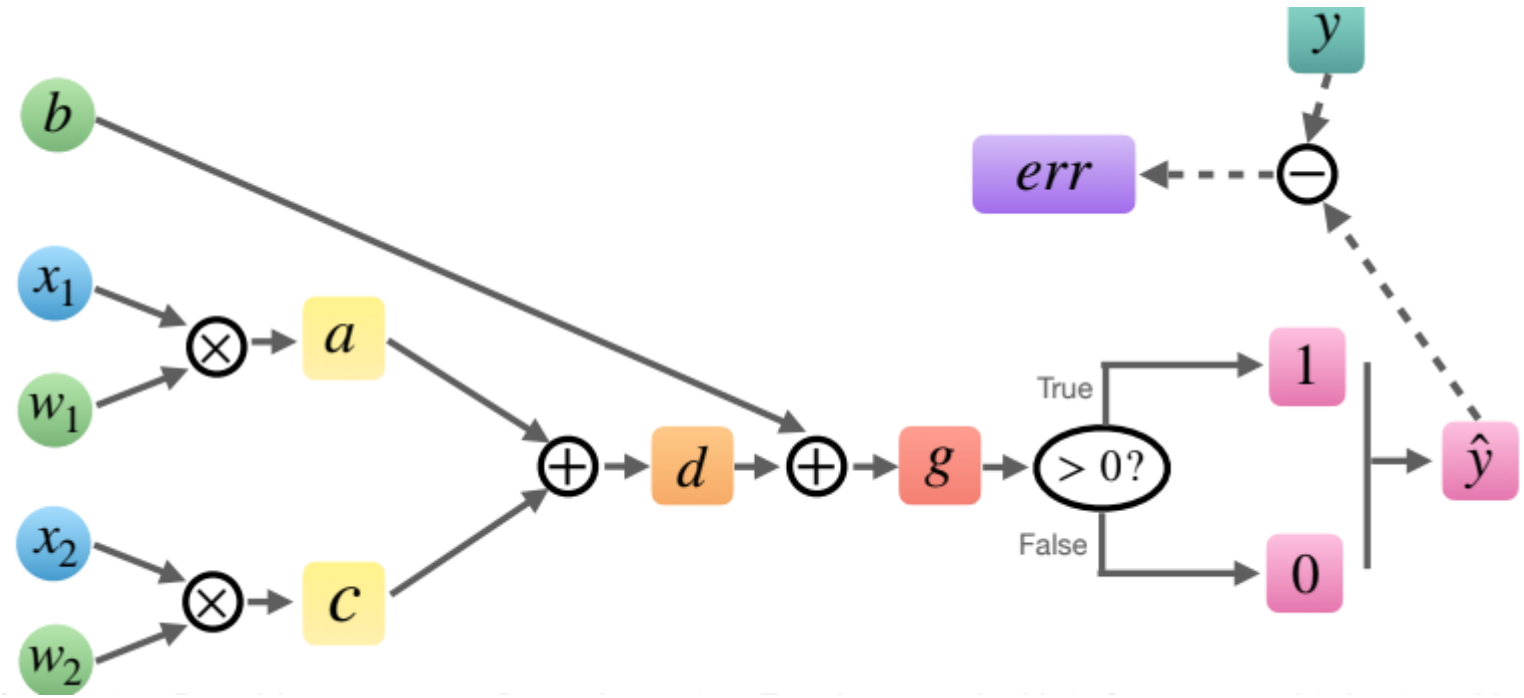


Función umbral

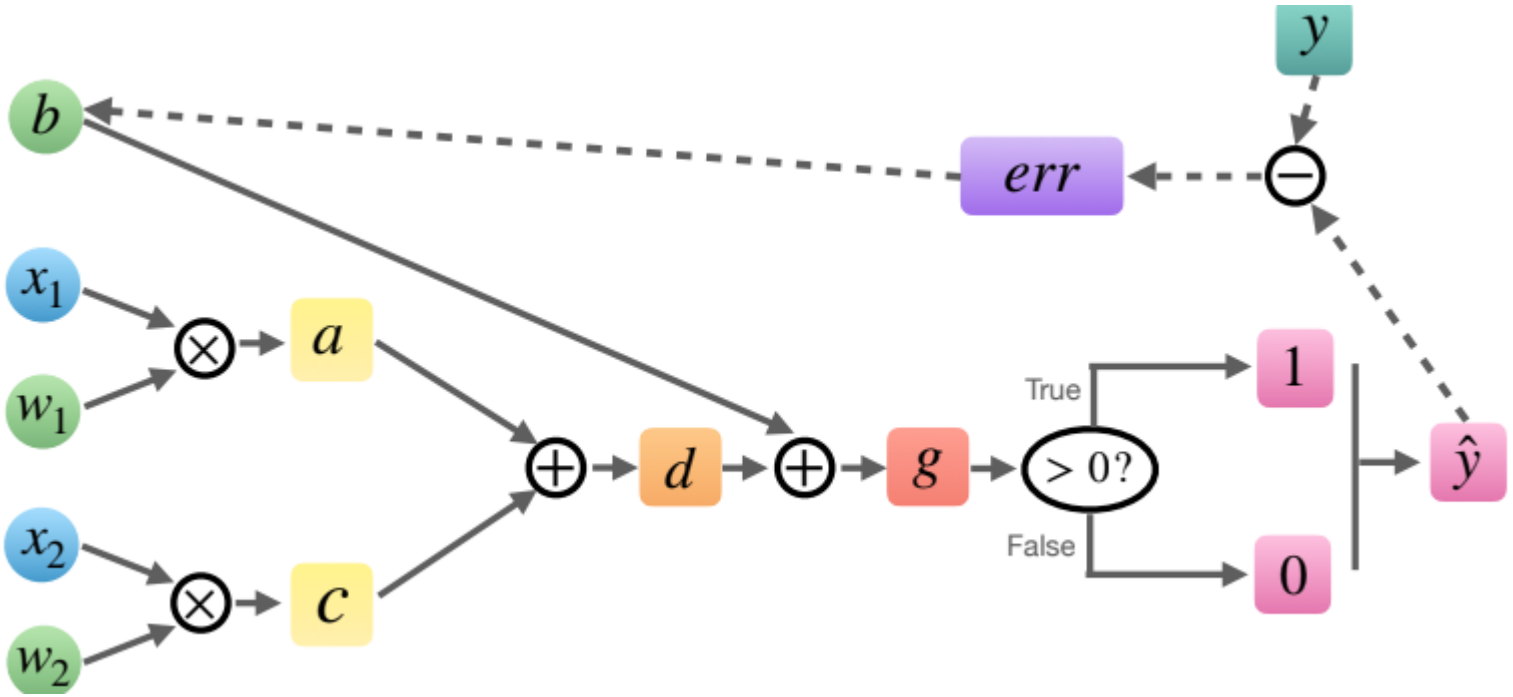


```python
class Perceptron:
    def __init__(self, num_features):
        self.num_features = num_features
        self.weights = torch.zeros(num_features)
        self.bias = torch.tensor(0.)

    def forward(self, x):
        weighted_sum_z = torch.dot(x, self.weights) + self.bias

        if weighted_sum_z > 0.:
            prediction = torch.tensor(1.)
        else:
            prediction = torch.tensor(0.)

        return prediction

    def update(self, x, true_y):
        prediction = self.forward(x)
        error = true_y - prediction

        # update
        self.bias += error
        self.weights += error * x

        return error
```

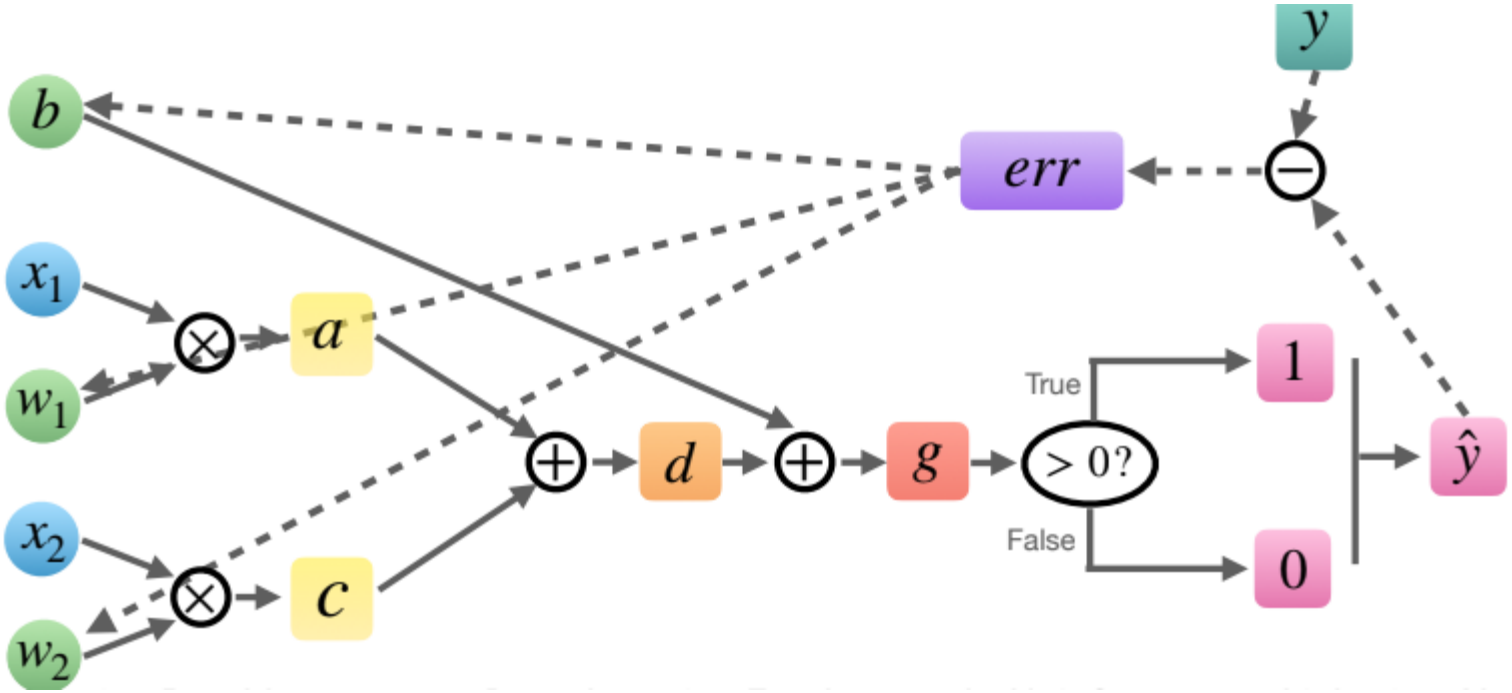
Cálculo del error



Actualizar los parámetros del modelo



Actualizar los parámetros del modelo



A continuación: Entrenamiento del modelo en PyTorch